Project - Low Level Design

on

Sports Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AANCHAL MARATHE	EN22CS301015
02	AAHMAD BURHANI	EN22CS301010
03	ABHISHEK SANKHALA	EN22CS301035
04	YASHASVI MODI	EN22CS3011114
05	AKSHAT YADAV	EN22CS301095

Group Name: 10D1

Project Number: GAI-10

Industry Mentor Name:

University Mentor Name: Nishant Shrivastava

Academic Year: 2025-2026

Table of Contents

1. Introduction
1.1. Scope of the document
1.2. Intended Audience
1.3. System overview
2. Architecture Diagram
3. System Design
3.1. Application Design
3.2. Process Flow
3.3. Information Flow
3.4. Components Design
3.5. Key Design Considerations
3.6. API Catalogue
3. References

1. Introduction

The Sports Planning Assistant Agent is an intelligent system built using Agentic AI concepts to assist users in analysing sports data, generating insights, and planning strategies. The system allows users to enter a high-level goal such as analysing team performance, generating a training plan, or retrieving match statistics.

The system automatically detects the sport, league, team, and relevant statistics from the user query. It uses multiple AI agents that collaborate to complete the task. These agents retrieve real-time sports data using APIs and generate insights along with visualizations such as charts and graphs.

The system is implemented using Python with frameworks such as CrewAI, LangChain, and a web interface built with Streamlit.

1.1 Scope of the Document

This document describes the system architecture, design components, workflow, and API integrations used in the Sports Planning Assistant Agent system.
The scope includes:

- Application architecture
- Agent-based system design
- Data flow within the system
- API integration for sports data
- Design considerations for scalability and usability

This document serves as a technical reference for developers, project reviewers, and system maintainers.

1.2 Intended Audience

This document is intended for:

- Software developers working on the project
- System architects
- Academic evaluators and instructors
- Students studying Agentic AI systems
- Anyone interested in understanding the system design and implementation

1.3 System Overview

The Sports Planning Assistant Agent is a multi-agent system where several intelligent agents collaborate to process user requests.

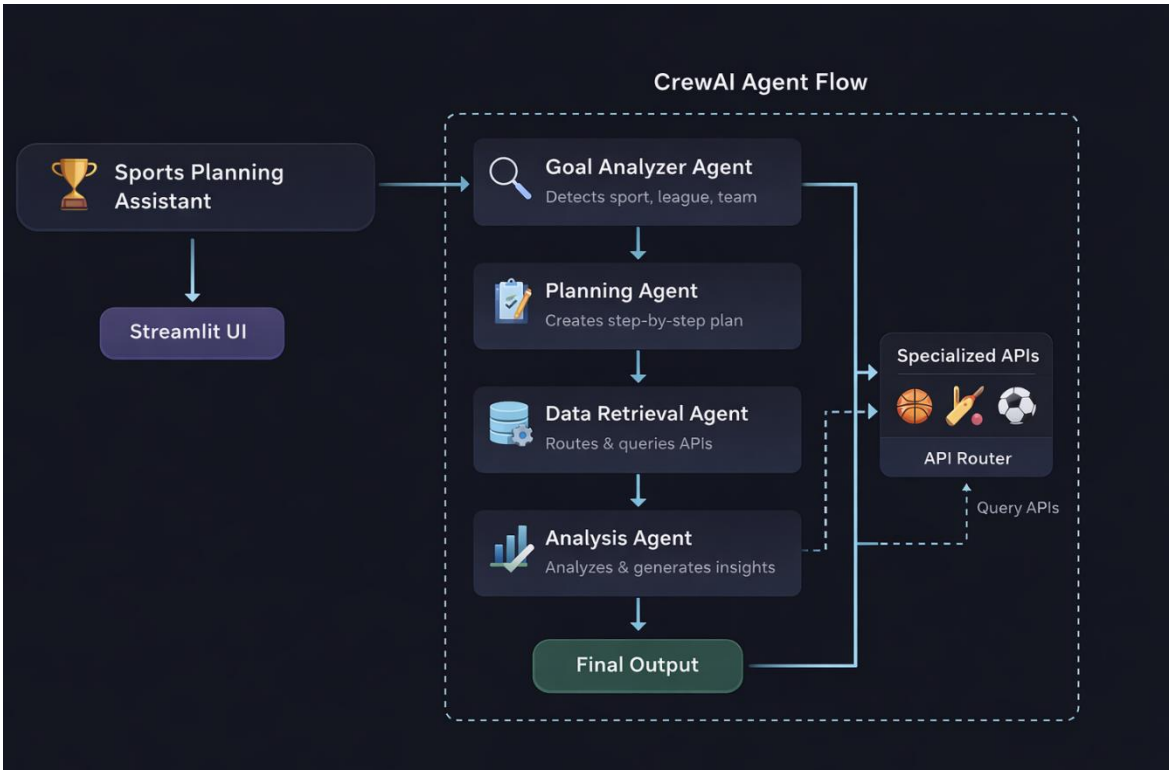
The system workflow includes:

1. The user enters a sports-related goal through the Streamlit interface.

2. The Goal Analyzer Agent interprets the request and extracts sports information.
3. The Planning Agent creates a step-by-step plan.
4. The Data Retrieval Agent collects sports statistics using APIs.
5. The Analysis Agent processes the data and generates insights.

This system demonstrates how Agentic AI can automate sports data analysis and decision-making.

2. Architecture Diagram



3. System Design

The system design describes the architecture, processes, and components used to build the Sports Planning Assistant Agent.

3.1 Application Design

The application follows a multi-layer architecture consisting of:

Presentation Layer: The user interface built using Streamlit allows users to input sports-related queries and view the analysis results.

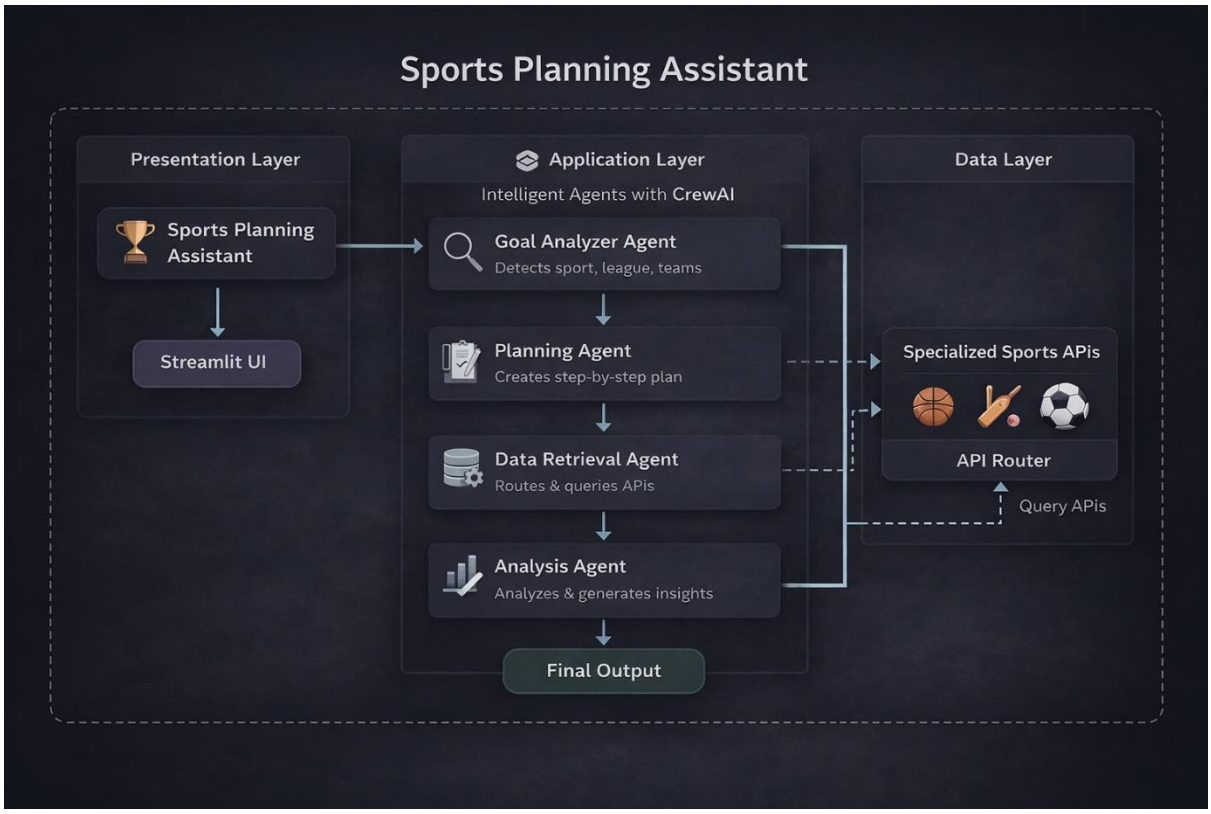
Application Layer: This layer contains the intelligent agents that process user goals and coordinate tasks.

Agents include:

- Goal Analyzer Agent
- Planning Agent
- Data Retrieval Agent
- Analysis Agent

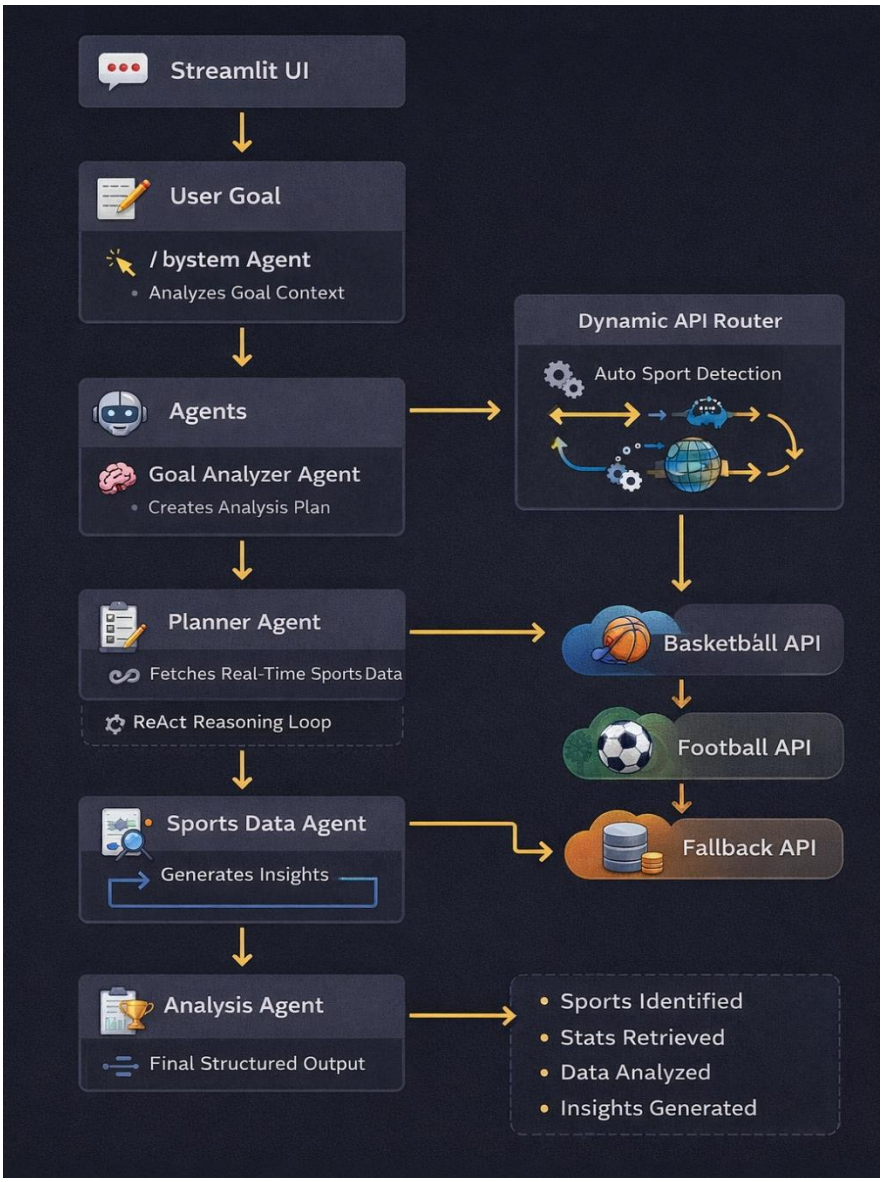
These agents are orchestrated using CrewAI.

Data Layer: This layer interacts with external sports APIs to retrieve real-time data.



3.2 Process Flow

The system follows the process flow below:

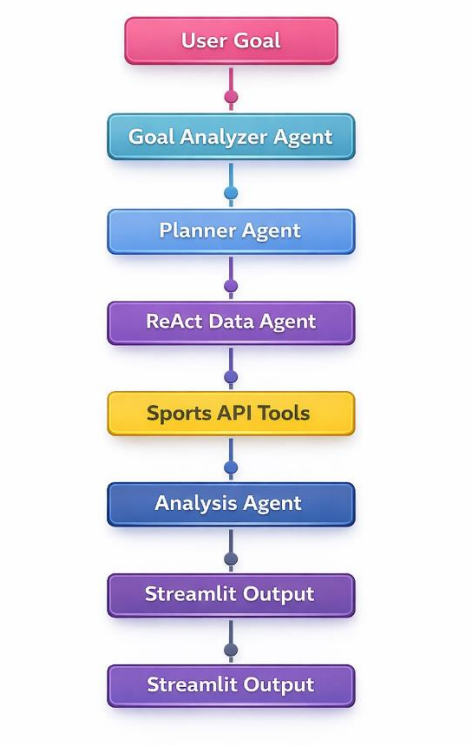


3.3 Information Flow

Information flows through the system in the following way:

1. The user provides a goal or query.
2. The Goal Analyzer Agent extracts key entities such as sport, league, and team.
3. The Planning Agent creates an execution plan based on the goal.
4. The Data Retrieval Agent fetches data from sports APIs.
5. The Analysis Agent processes the data and generates insights.
6. The final output is displayed to the user.

This structured flow ensures efficient processing of user queries.



3.4 Components Design

The system is composed of several components.

1. Streamlit Interface: Handles user interaction and displays the results.

2. Goal Analyzer Agent: Extracts sports-related entities from the user goal like identifying the sports, leagues, format.

3. Planning Agent: Breaks the user goal into smaller executable tasks.
Functions:

- Create an execution plan for sports analysis

4. Data Retrieval Agent: Fetches sports data from external APIs and add reasoning factor to the system. Functions:

- Fetch team data
- Fetch Sports Data

5. Analysis Agent: Processes the retrieved data and generates insights like team performances and statistics.

3.5 Key Design Considerations

Several factors were considered during system design.

- **Scalability:** The multi-agent architecture allows the system to support additional sports and APIs in the future.

- **Modularity:** Each agent performs a specific task, making the system modular and easier to maintain.
- **Extensibility:** New tools and APIs can be integrated without major changes to the system.
- **Usability:** The Streamlit interface provides a simple and user-friendly experience.
- **Performance:** The system uses efficient API calls and structured workflows to reduce response time.

3.6 API Catalogue

The system integrates external sports APIs to retrieve data.

Example APIs used: **Sports Data API**

Provides information such as:
- Team details
- League information
- Player statistics
- Match data

Example endpoint:
https://www.thesportsdb.com/api/v1/json/3/searchteams.php?t=TeamName

Functions using APIs:
- fetch_team_data()
- fetch_league_data()

These APIs allow the system to access real-time sports statistics.

4. References

1. CrewAI Official Documentation
2. LangChain Documentation
3. Sports API Documentation (TheSportsDB)
4. Research papers on Agentic AI systems and multi-agent architectures